

Adventist University of Central Africa
Department of Information Technology

Introduction to Big Data, Final Project

NBA Game Analytics, Win Prediction, and AI Assistant

Confiance UFITAMAHORO
Student ID: 27185

Dataset: NBA games and teams (2021–2023 seasons)

Source: balldontlie API (free tier)

GitHub: <https://github.com/confiance22/27185-nba-analytics-ml-ai-assistant>

Deployed version link: <https://27185-nba-analytics-ml-ai-assistant.up.railway.app/>

1. Overview

For my final project I moved away from the WHO health data I used in Assignment 5 and worked with NBA basketball data instead, pulled from the `balldontlie` API. My dataset covers every completed regular-season and postseason game from the 2021, 2022, and 2023 seasons, along with the 30 currently active NBA teams.

The question I wanted to answer was simple: can I predict which team will win a given NBA game using only information that would have been known before the game started, specifically, how well each team had been playing coming into that matchup? Basketball is known for having a real home-court advantage, so I also wanted to see how strong that effect actually is in the data, rather than just assuming it.

Beyond the prediction question, this project also required me to build a full pipeline around that data: cleaning and validating it, exploring it, training and comparing machine learning models, building an interactive Tableau dashboard, and building a small AI assistant that can answer plain-English questions about the data. I also set up my ETL pipeline to run automatically on a schedule, which is described in Section 6.

2. How I Cleaned and Checked My Data

Raw data first, always

Every pipeline run stores the exact, untouched API response first, in `raw_teams` and `raw_games` tables, before anything is cleaned. If my cleaning logic ever has a bug, I can fix it and re-run the cleaning step against this raw data without hitting the API again. This is the same raw/cleaned split I used in Assignment 5, and it proved useful more than once while I was debugging.

Teams: removing 15 teams that shouldn't be there

The `/teams` endpoint returned 45 teams, not 30. The extra 15 were defunct 1940s franchises that no longer exist, and I noticed all of them had empty conference and division fields. I used that as the validation rule, keeping teams only if both fields are filled in, which correctly kept 30 teams and rejected 15.

Games: five checks before a game counts as valid

For games, I check five things before a row is allowed into the cleaned table:

- The game's status must be "Final", this rejects games that are scheduled or still in progress.
- The game must not be marked as postponed.
- Both the home and visitor score must be present and be valid, non-negative numbers.
- The two scores can't both be zero, a 0-0 "final" score would actually mean the game never started, not that it ended in a scoreless tie (basketball doesn't have 0-0 finals).
- The home team ID and visitor team ID must be different, a team can't play itself.

Against the real 3,962 games, every game passed all five checks with zero rejected. I confirmed this rather than trusting it by querying the raw status and postponed values directly, which all showed "Final" and not postponed, as expected for fully completed seasons.

What could go wrong, and how I handled it

I hit two real problems. First, a network drop partway through a 40-page extraction lost all progress because nothing was saved until the end; I fixed this by saving a partial file after every page, so a crash now costs at most one page of re-work. Second, the API returned HTTP 429 rate-limit errors during a cloud run, and my short retry waits (1, 2, 4 seconds) were too short for a limit that resets after a full minute; I gave 429 errors their own longer wait (up to 60 seconds) instead of treating them like any other error.

3. Exploratory Data Analysis

Before building any models, I looked at the cleaned dataset as a whole: 30 teams and 3,962 games, with zero missing values and zero duplicate rows in either table. This wasn't an accident, it's a direct result of the validation checks described above catching problems before they ever reached the cleaned tables.

What I found looking at individual columns

- Combined score per game (home + visitor points) has a mean around 225 points, ranging from as low as 143 to as high as 351 in a single game.
- 2,214 games were won by the home team versus 1,748 won by the visitor, a home win rate of about 55.9% across the whole dataset.
- Games are split almost exactly evenly across the three seasons (1,323 / 1,320 / 1,319), and 259 of the 3,962 games were postseason games.
- Teams are evenly split 15/15 between the Eastern and Western conferences, and 5 teams per division across all six divisions.

Comparing two columns at once

Looking at how average combined score changed across seasons, I saw scoring go up somewhat: 220.6 points per game in 2021, rising to 228.7 in 2022, then settling slightly at 227.3 in 2023. It's a modest change, not a dramatic trend, but noticeably higher after 2021.

I also compared home win rate by conference, since I was curious whether the home-court advantage was the same across the league. It wasn't quite: home teams in the Eastern Conference won 54.7% of their home games, while home teams in the Western Conference won 57.0%, a real but fairly small gap.

What I take away from this

The clearest finding from the EDA is that home-court advantage is real in this dataset, sitting just under 56% overall. That number matters a lot for the modeling section below, because any model I build needs to beat that number meaningfully to prove it learned something beyond the obvious home-court bias.

4. Machine Learning

Features I engineered

Since the free tier of the API only gives team-level game results (no player stats, no box scores), I couldn't use anything like recent player performance. Instead, I engineered two "team form" features for both the home and visitor team going into each game:

- Season win rate so far, that team's win rate in games earlier in that same season, before this game. Resets to a neutral 0.5 at the start of each season.

- Last-5 win rate, that team's win rate over their last 5 games before this one, regardless of season boundary.

I was careful to only ever use information from games that happened strictly before the game being predicted, never information from that game itself or any future game. This matters because using future information to "predict" the past is a classic and easy-to-miss mistake in this kind of feature engineering.

Models and scores

I trained and compared two classification models predicting home_win (1 if the home team won, 0 otherwise), using an 80/20 train-test split with the class balance preserved in both sets:

Model	Accuracy	Notes
Baseline (always predict home win)	55.9%	What you'd get just guessing home team every time
Logistic Regression	66.6%	Best performer
Random Forest (max_depth=6)	63.8%	Slightly behind Logistic Regression

Both models clearly beat the 55.9% baseline, so they learn real patterns from team form, not just home-court bias. Logistic Regression scored better because my win-rate features are smooth, roughly linear signals, which is exactly what linear models are good at, while Random Forest found little non-linear structure to exploit with only four simple features. Both models were better at detecting home wins (~83-86% recall) than visitor wins (~40-42%), meaning they rely more on the general home-court pattern than on confidently separating upsets, a real limitation.

Checking for overfitting and underfitting

I trained the Random Forest model at three different max_depth settings to see how training accuracy compared to test accuracy at each one:

max_depth	Train Accuracy	Test Accuracy	Gap
2 (very shallow)	62.2%	64.6%	-2.4 pts (underfitting — too simple to use the signal well)
6 (chosen)	67.3%	63.8%	3.5 pts (small, healthy gap)
None (unlimited)	98.9%	61.2%	37.7 pts (severe overfitting)

The table shows max_depth=6 was the right choice: at depth 2 the model is too simple to use the signal, while unlimited depth memorizes the training games (98.9% train accuracy) and then performs worse on new games, a clear sign of overfitting. Depth 6 sits in between, with training and test accuracy close together.

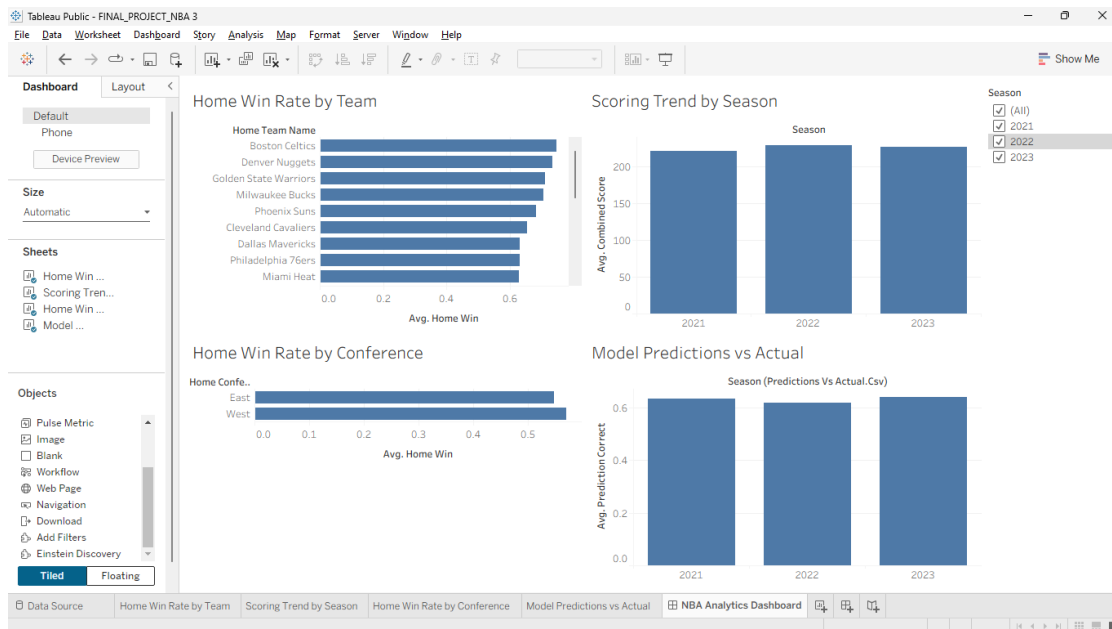
5. Tableau Dashboard

My dashboard has four charts and two interactive filters, built from the cleaned games and teams data plus my saved model's predictions.

- Home Win Rate by Team, bar chart ranking all 30 teams by their average home win rate.
- Scoring Trend by Season, combined score (home + visitor points) averaged by season, showing the 2021 → 2022 → 2023 trend described in the EDA section.

- Home Win Rate by Conference, East vs West comparison, matching the 54.7% / 57.0% split from the EDA.
- Model Predictions vs Actual, shows my Logistic Regression model's prediction accuracy broken down by season, satisfying the requirement that at least one chart shows a result from the machine learning work.

Both filters (season, and home team conference) are set to apply across all four charts at once, so selecting a single season or conference updates the whole dashboard together.



Public link:

https://public.tableau.com/app/profile/confiance.ufitamahoro/viz/FINAL_PROJECT_NBADSHBRD/NBAAnalyticsDashboard

6. AI Assistant

My AI assistant is a Python program that runs both as a command-line tool and as a Flask web app, letting someone type a plain-English question about the NBA data and get a plain-English answer back. It uses Groq (running the Llama 3.1 model) to do the whole task end-to-end:

- The Llama 3.1 model turns the user's question into a SQL query, using a description of my exact table structure so it doesn't invent columns that don't exist.
- The SQL query runs against my `cleaned_games` and `cleaned_teams` tables.
- The same model takes the SQL result and turns it into a natural, plain-English sentence.

Handling a question that can't be answered

Question: "do we have list of player in lakers"

Answer: "I can only answer questions about NBA teams and games from the 2021-2023 seasons, that question is outside what I can look up."

My schema only contains team and game data, no player-level information exists anywhere in it. The model correctly recognized this was outside what the schema could answer and returned a special

UNANSWERABLE signal, which my code catches and turns into a graceful, honest message instead of guessing at an answer or crashing.

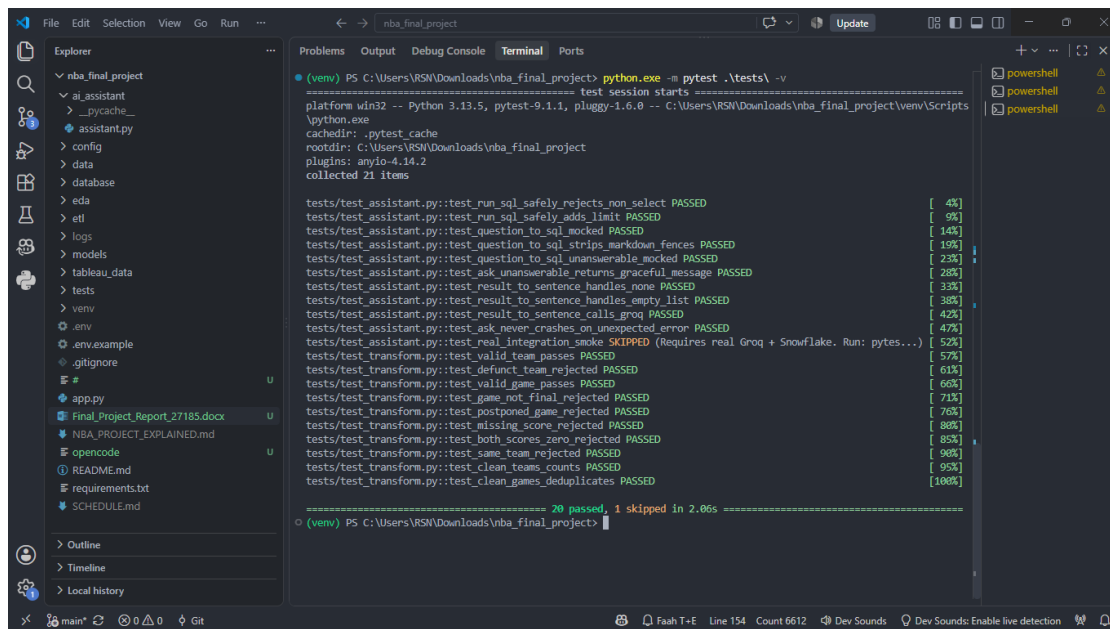
A limitation I found through testing

While testing, I asked a head-to-head question like "between cleveland and lakers, which team do you think will win". The assistant didn't crash, but the SQL it generated redefined what home_win means inside a subquery, and it returned a confidently-stated but meaningless "tie, 0-0" answer. This showed me that LLM-generated SQL can be perfectly valid yet wrong without ever throwing an error, which is harder to catch than a crash. I improved my prompt to be clearer about what home_win means and not to redefine it in subqueries, which reduced but didn't fully remove this risk for complex comparative questions.

7. Testing

I wrote 20 automated tests with pytest. The 10 tests for the data cleaning logic check that valid teams and games are kept while bad ones (defunct teams, non-final, postponed, missing scores, impossible 0-0 finals, a team playing itself) are rejected, plus correct counting and deduplication.

The 10 AI assistant tests check that dangerous SQL (like DROP or DELETE) is rejected, a missing LIMIT is added safely, unanswerable questions get a graceful reply, and unexpected errors never crash the program. I used Python's mocking tools to fake the Groq API, so the tests are fast, free, and reliable. One extra end-to-end test calls the real APIs but is skipped by default.



```
(venv) PS C:\Users\RSN\Downloads\nba_final_project> python.exe -m pytest .\tests\ -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\RSN\Downloads\nba_final_project\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\RSN\Downloads\nba_final_project
plugins: anyio-4.14.2
collected 21 items

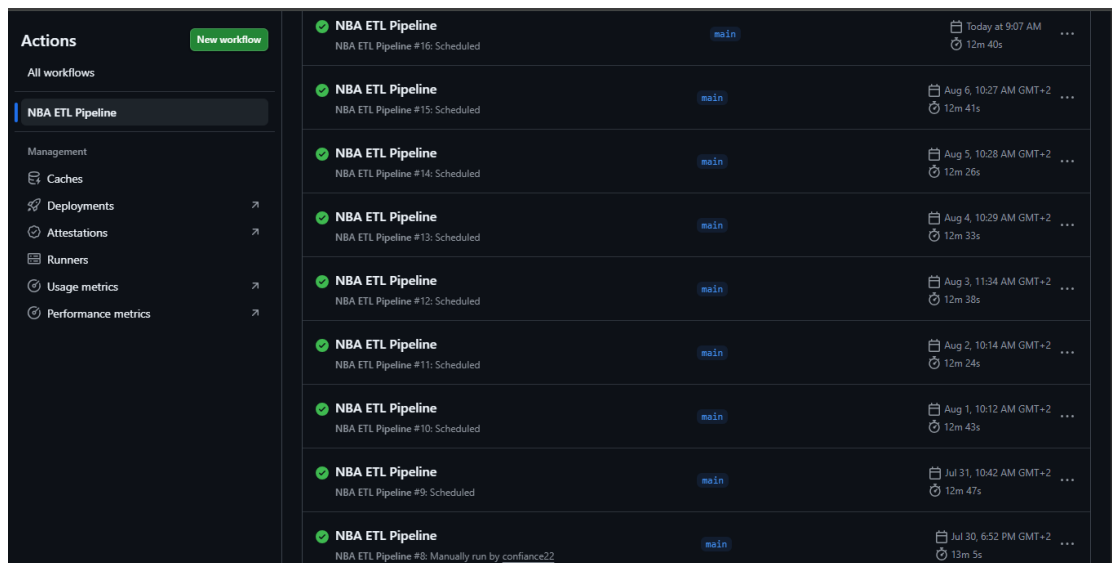
tests/test_assistant.py::test_run_sql_safely_rejects_non_select PASSED [ 4%]
tests/test_assistant.py::test_run_sql_safely_adds_limit PASSED [ 9%]
tests/test_assistant.py::test_question_to_sql_mocked PASSED [ 14%]
tests/test_assistant.py::test_question_to_sql_strips_markdown_fences PASSED [ 19%]
tests/test_assistant.py::test_question_to_sql_unanswerable_mocked PASSED [ 23%]
tests/test_assistant.py::test_ask_unanswerable_returns_graceful_message PASSED [ 28%]
tests/test_assistant.py::test_result_to_sentence_handles_none PASSED [ 33%]
tests/test_assistant.py::test_result_to_sentence_handles_empty_list PASSED [ 38%]
tests/test_assistant.py::test_result_to_sentence_calls_groq PASSED [ 42%]
tests/test_assistant.py::test_ask_never_crashes_on_unexpected_error PASSED [ 47%]
tests/test_assistant.py::test_real_integration_smoke SKIPPED (Requires real Groq + Snowflake. Run: pytes...) [ 52%]
tests/test_transform.py::test_valid_team_passes PASSED [ 57%]
tests/test_transform.py::test_defunct_team_rejected PASSED [ 61%]
tests/test_transform.py::test_valid_game_passes PASSED [ 66%]
tests/test_transform.py::test_game_not_final_rejected PASSED [ 71%]
tests/test_transform.py::test_postponed_game_rejected PASSED [ 76%]
tests/test_transform.py::test_missing_score_rejected PASSED [ 80%]
tests/test_transform.py::test_both_scores_zero_rejected PASSED [ 85%]
tests/test_transform.py::test_same_team_rejected PASSED [ 90%]
tests/test_transform.py::test_clean_teams_counts PASSED [ 95%]
tests/test_transform.py::test_clean_games_deduplicates PASSED [100%]

===== 20 passed, 1 skipped in 2.06s =====
(venv) PS C:\Users\RSN\Downloads\nba_final_project>
```

8. Automated Deployment

A cron job is a task that runs automatically on a server at a fixed time, without anyone starting it by hand. I set up a cron job on GitHub Actions that runs my ETL pipeline every day (and can be triggered manually). The database is Snowflake, hosted in the cloud, so the pipeline runs without my computer being on.

Setting this up wasn't trivial. I first tried Supabase, but GitHub's cloud runners could not connect to it because of an IPv6 mismatch, so I moved to the Snowflake database, using its connector and SQL syntax (like MERGE INTO) so loads can be re-run safely without creating duplicates.



The screenshot shows the GitHub Actions interface for a workflow named 'NBA ETL Pipeline'. On the left, a sidebar contains a 'New workflow' button and a list of management options: Caches, Deployments, Attestations, Runners, Usage metrics, and Performance metrics. The main area displays a list of workflow runs, each with a green checkmark indicating success. The runs are labeled 'NBA ETL Pipeline' followed by a number (e.g., #16, #15, #14, #13, #12, #11, #10, #9, #8) and the word 'Scheduled'. Each run entry includes a 'main' branch label, a calendar icon showing the run time (e.g., 'Today at 9:07 AM', 'Aug 6, 10:27 AM GMT+2'), and a clock icon showing the duration (e.g., '12m 40s', '12m 41s'). The bottom-most run, #8, is noted as 'Manually run by confidence22' and occurred on 'Jul 30, 6:52 PM GMT+2' with a duration of '13m 5s'.

Run ID	Branch	Run Time	Duration
NBA ETL Pipeline #16: Scheduled	main	Today at 9:07 AM	12m 40s
NBA ETL Pipeline #15: Scheduled	main	Aug 6, 10:27 AM GMT+2	12m 41s
NBA ETL Pipeline #14: Scheduled	main	Aug 5, 10:28 AM GMT+2	12m 26s
NBA ETL Pipeline #13: Scheduled	main	Aug 4, 10:29 AM GMT+2	12m 33s
NBA ETL Pipeline #12: Scheduled	main	Aug 3, 11:34 AM GMT+2	12m 38s
NBA ETL Pipeline #11: Scheduled	main	Aug 2, 10:14 AM GMT+2	12m 24s
NBA ETL Pipeline #10: Scheduled	main	Aug 1, 10:12 AM GMT+2	12m 43s
NBA ETL Pipeline #9: Scheduled	main	Jul 31, 10:42 AM GMT+2	12m 47s
NBA ETL Pipeline #8: Manually run by confidence22	main	Jul 30, 6:52 PM GMT+2	13m 5s

9. What I Would Improve With More Time

- Add player-level statistics if I upgrade past the free API tier, to improve prediction accuracy beyond team-level win rates.
- Add more advanced ML features, such as head-to-head history between two teams and rest days between games, which are known to affect real NBA outcomes.
- Improve the AI assistant's handling of head-to-head and comparative questions, which can produce valid-looking but wrong SQL rather than a clear error.
- Track actual playoff series and championship outcomes, not just individual game results, so questions like "who won the playoffs" can be answered correctly.